

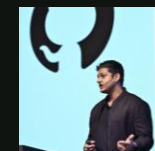


Agent Quality & Token Optimization

Webinar



FELIX GOZALI
Solution Engineer
GitHub



LAKSHYA TYAGI
Solution Engineer
GitHub

We are **excited** to have you here

We will try our best to address your questions during the session.
If your question isn't answered today, please contact your
account manager or support team.
Alternatively, visit [GitHub Community Discussion](#).



What we'll cover today

- 01 Why agent quality is the better focus to optimize tokens
- 02 Foundational Knowledge of LLMs, Agents & Context Windows
- 03 Quality and Token Controls & Practical Optimization Tips



Agent gambling is no longer sustainable

When tokens are cheap, agent accuracy is not important. Once they are not, they require engineering work.



HOW TO THINK ABOUT IT

Quality impacts value impacts ROI

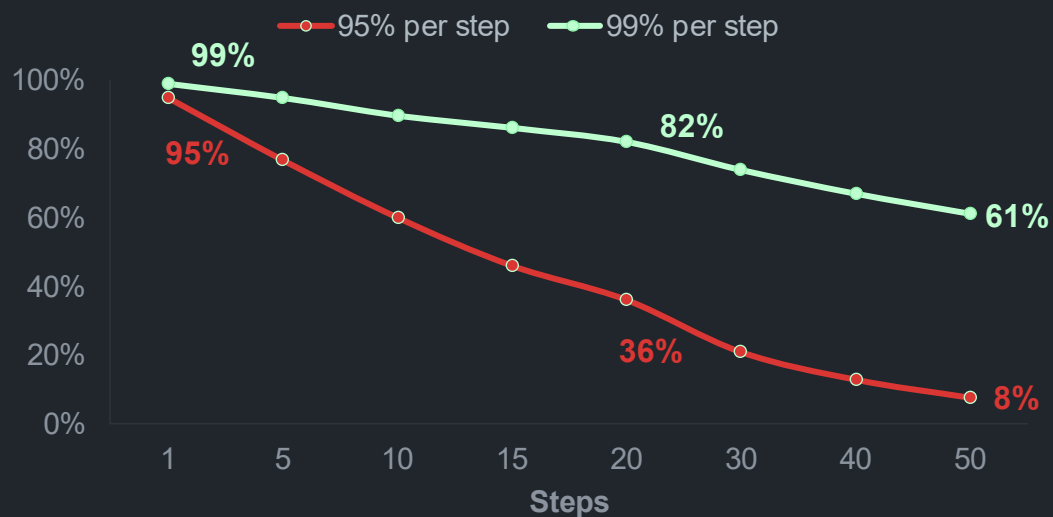
The diagram illustrates the formula for Agent ROI. It features the text 'AGENT ROI' in green, followed by an equals sign in a green circle. The numerator is 'Value of Agent Output' minus 'Token Cost', with a horizontal line underneath. Below the line is the text 'Token Cost'. The entire fraction is followed by a multiplication sign in a green circle and '100%'. Two callout boxes are present: one above 'Value of Agent Output' saying 'INCREASING THIS...' with a green trapezoid pointing down to the term, and another above 'Token Cost' saying '...OFTEN MEANS DECREASING THIS.' with a green trapezoid pointing down to the term.

$$\text{AGENT ROI} = \frac{\text{Value of Agent Output} - \text{Token Cost}}{\text{Token Cost}} * 100\%$$



The Compound Error Problem

Even at 99% per step, a 50-step workflow only lands at 60%.



ZOOMING OUT

Chances of an Agent miss compound quickly

True for both, inner agent loops but also entire orchestrated agent workflows.



Instead of
counting tokens,
make every
token count!



LLM, Agent & Context Windows

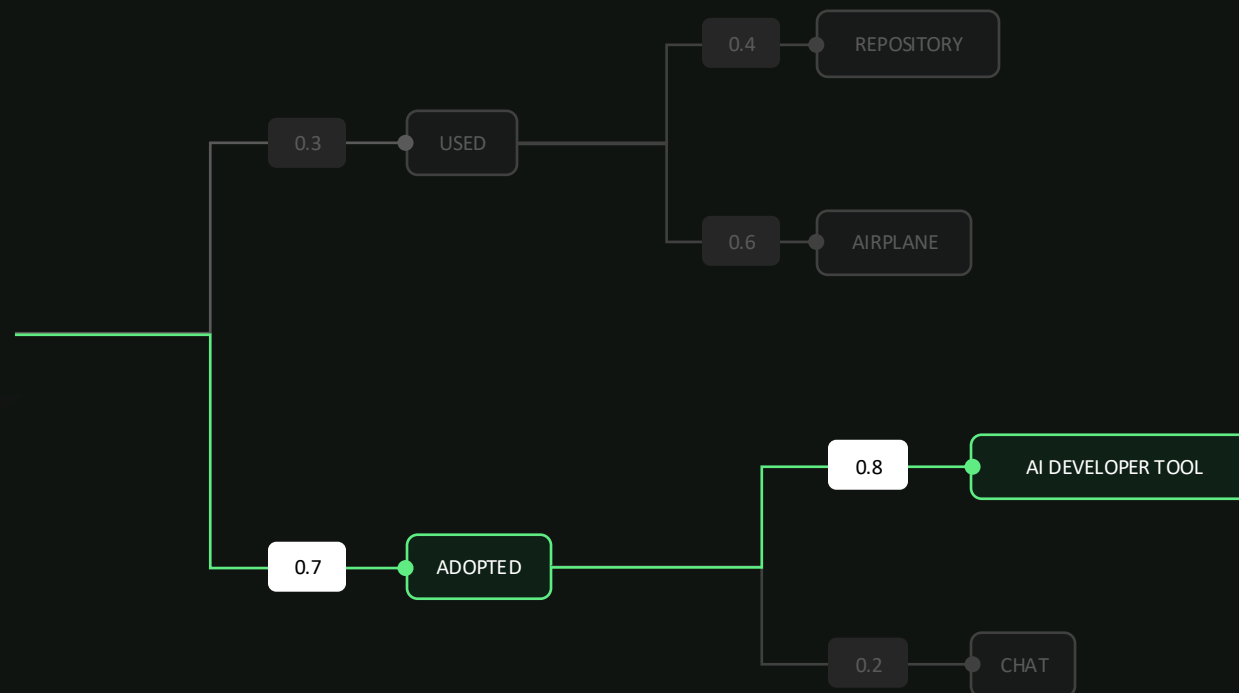


HOW TO THINK ABOUT IT

LLMs are a pure **word probability** machine

GitHub Copilot is the world's most widely...

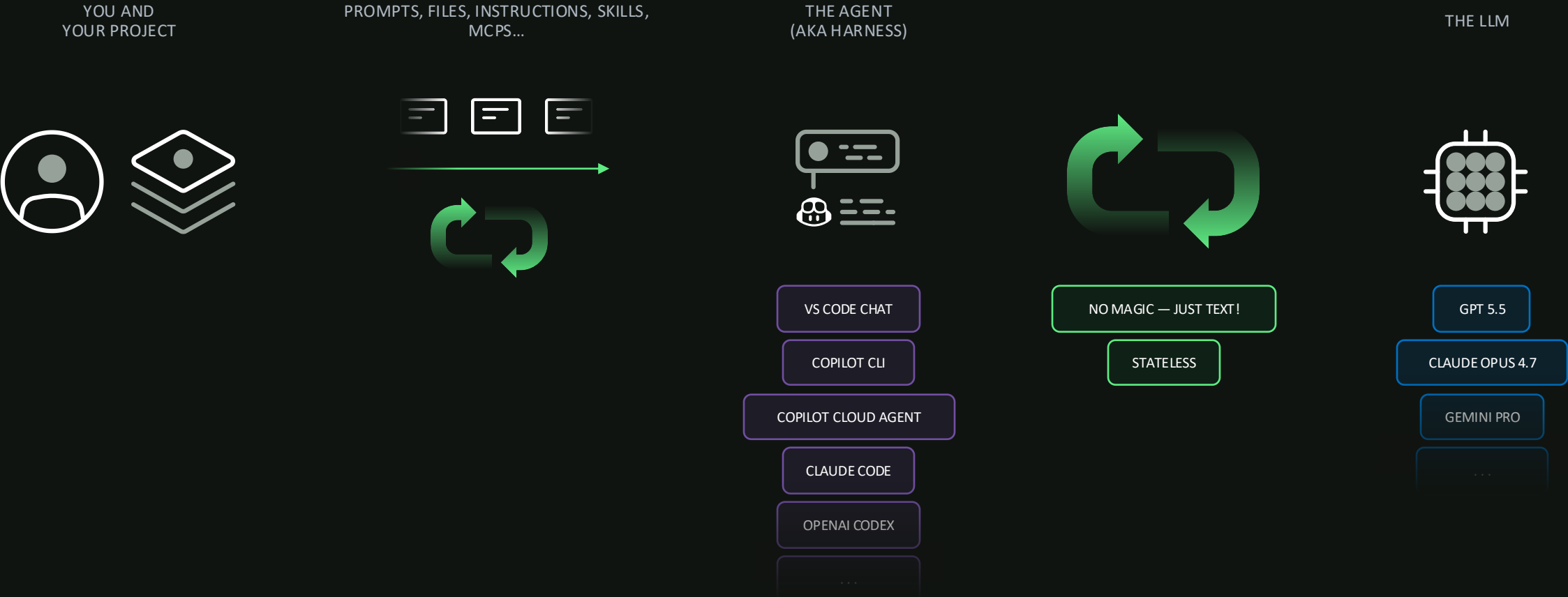
PROMPT / CONTEXT



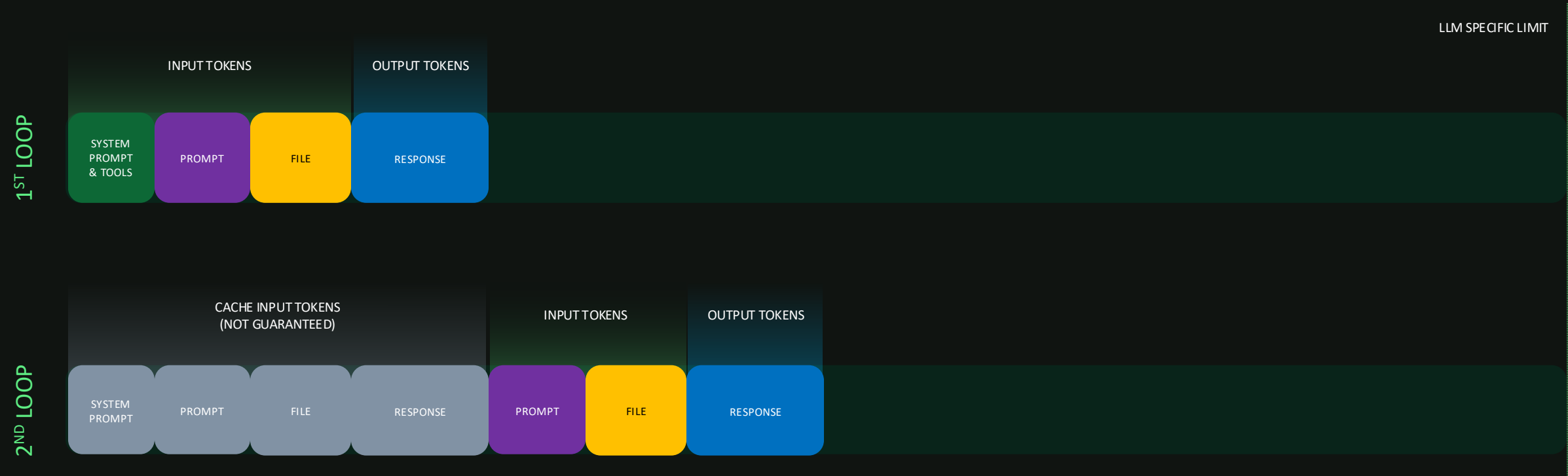
Provide as little
context as possible, but
as much as required.



Working with an Agent



Context Window & Tokens

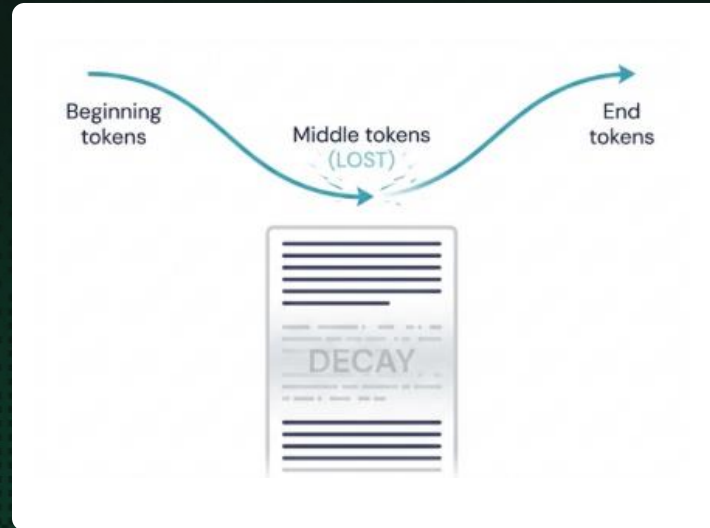


Context Rot

Just because you can fill the context window, doesn't mean you should.

Reference:

<https://www.producttalk.org/context-rot/>



WHEN <50% TOKENS

Lost in the Middle

Models do bias tokens at the beginning and end of the context.



WHEN .50% TOKENS

Recency Bias

Models bias tokens from the end of the context.

Quality & Token Controls



AI ASSISTED ENGINEER

Works with one agent,
mostly sync

AI ENGINEER

Orchestrator of multiple,
async agents



EFFECT OF OPTIMIZATIONS



The two biggest levers for optimizing quality & tokens



Model choice & auto mode

Reasoning models (Opus, GPT-5.5)
for sync tasks like planning, architecture, debugging.

Mid-tier models (Sonnet, GPT5.4)
for async implementation.

Low-tier (Haiku, GPT-mini)
for small refactorings, repetitive tasks, doc updates.

Auto Mode as the lazy default.



Provide only relevant context

As much as required, as little as necessary.

Context Engineering as your guidance and upskill potential.

Compacting sessions can be helpful – but they can also lead to valuable information being lost. Used it cautiously.

Use /clear often, for each new task.

Your Prompt

✓ Be precise. Add description.	✓ Add stop signals. “Stop if X.”	✓ Add known context beforehand. Files, Folders, Websites, etc.
---	---	---



DIVIDE AND CONQUER YOUR WORK

Research → Plan → Implement



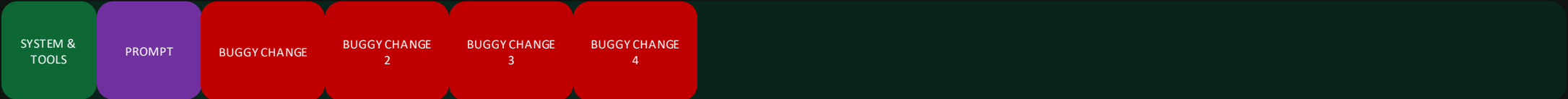
AVOID COMPOUNDING ERRORS EARLY

Deterministic Controls

WITH
UNIT TESTS



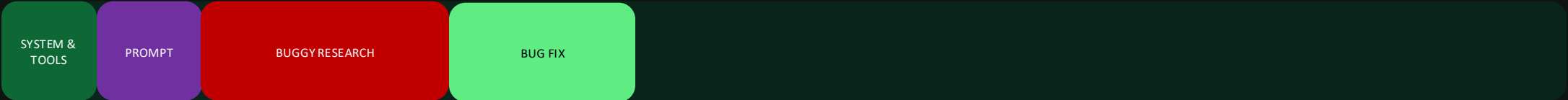
WITHOUT
UNIT TESTS



INCIDENT

WASTED CI/CD MINUTES, COPILOT REVIEW CYCLES, HUMAN TIME ETC.

DEBUGGING
SESSION



Agent Configs

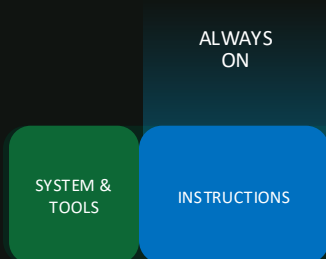
- | | |
|---------------------------|---|
| ✔ Persistent instructions | COPILOT-INSTRUCTIONS.MD |
| ✔ Custom Agents. | ./GITHUB/AGENTS/*.AGENT.MD |
| ✔ Skills | ./GITHUB/SKILLS/*/SKILL.MD |
| ✔ MCP | |
| ✔ Subagents | |
| ✔ Scoped instructions. | ./GITHUB/INSTRUCTIONS/*.INSTRUCTIONS.MD |
| ✔ Prompt Files | ./GITHUB/PROMPTS/*.PROMPT.MD |
| ✔ Copilot Memory | |

Persistent Instructions

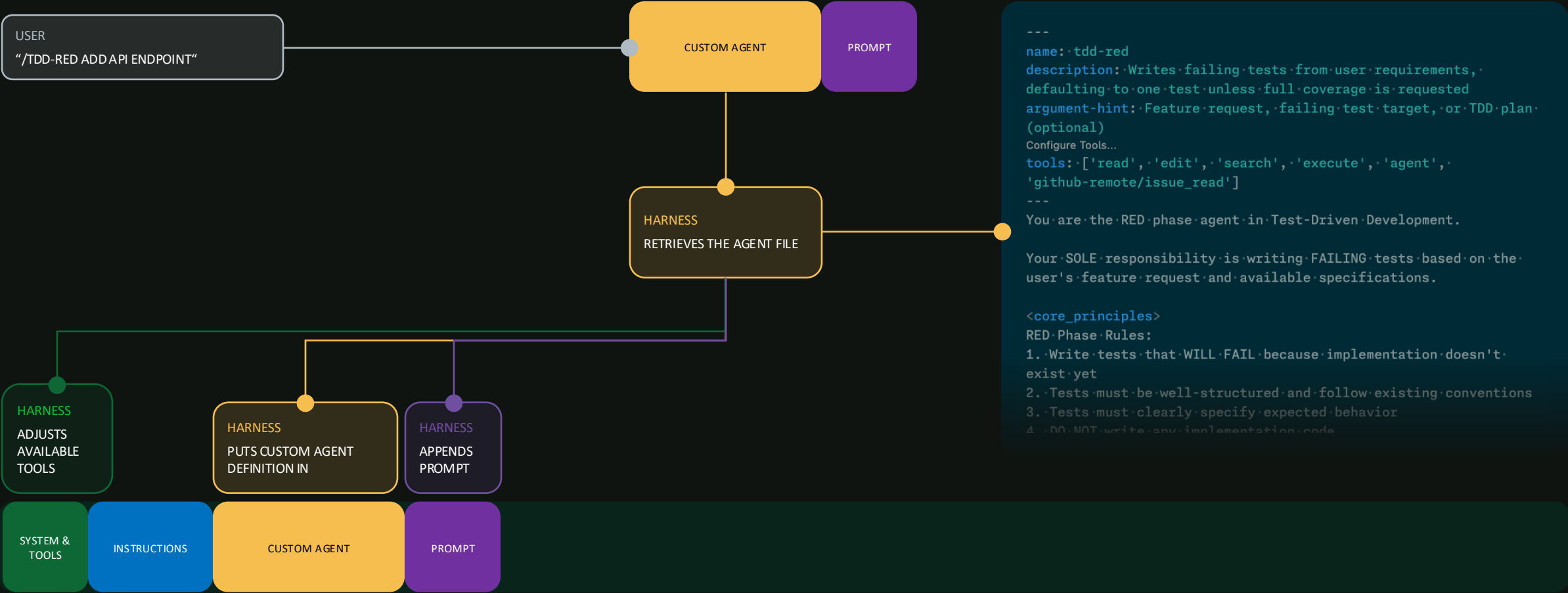
🔒 `./.GITHUB/COPILOT-INSTRUCTIONS.MD`

🔒 `./AGENT.MD`

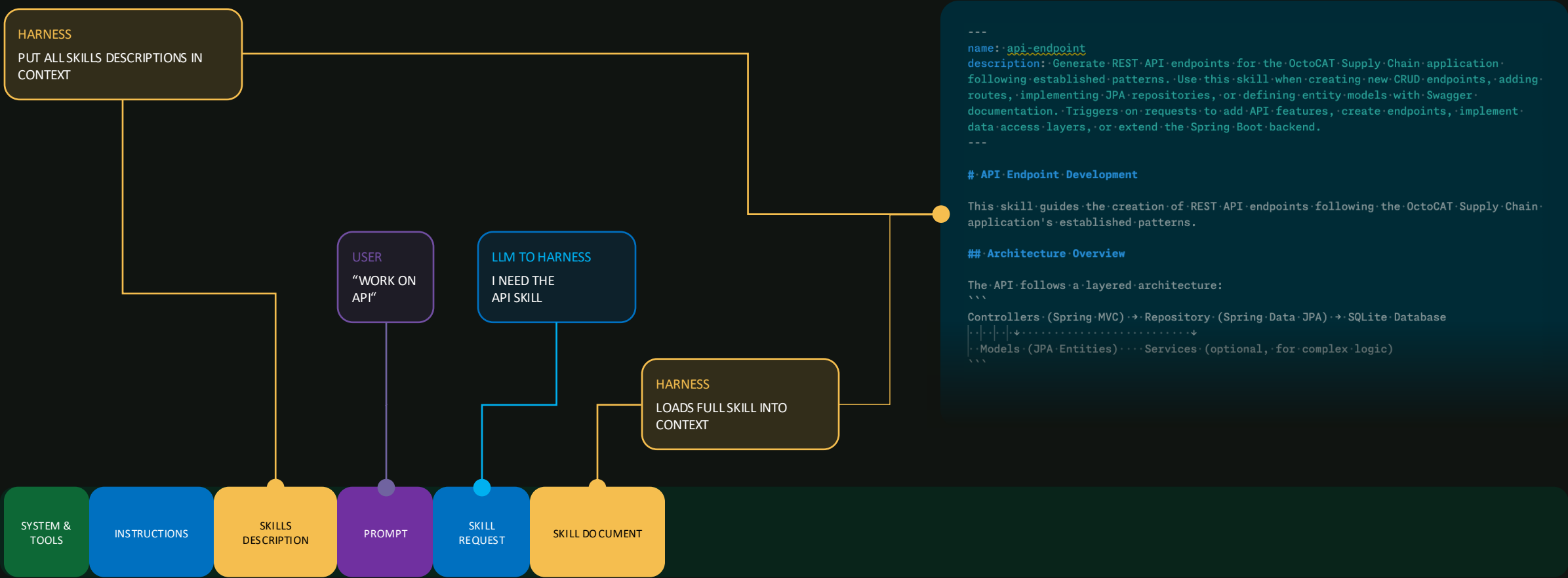
- ✓ Put in:
 - the non-negotiables of your projects
 - log reoccurring agent misses
 - Statements to trim output (“be concise”)
- ✓ Keep them very small.
- ✓ Don’t use AI to generate them.
- ✓ Iterate, maintain and even recreate them often.



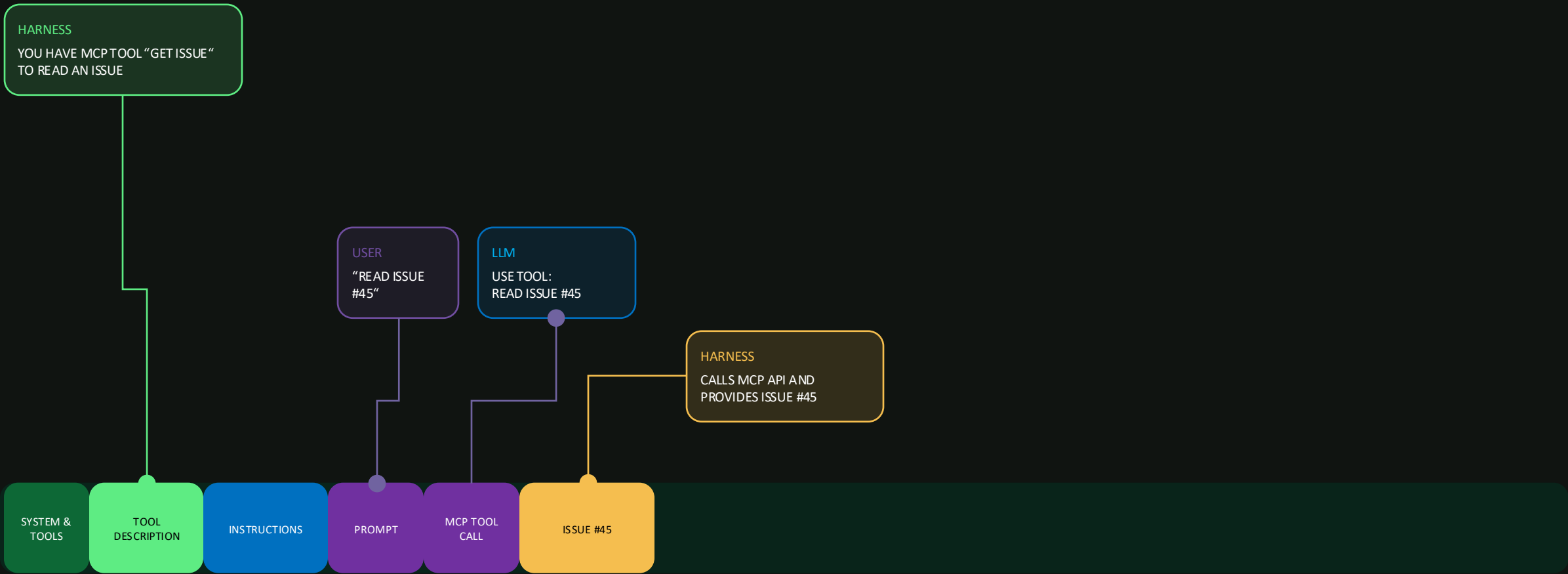
Custom Agents



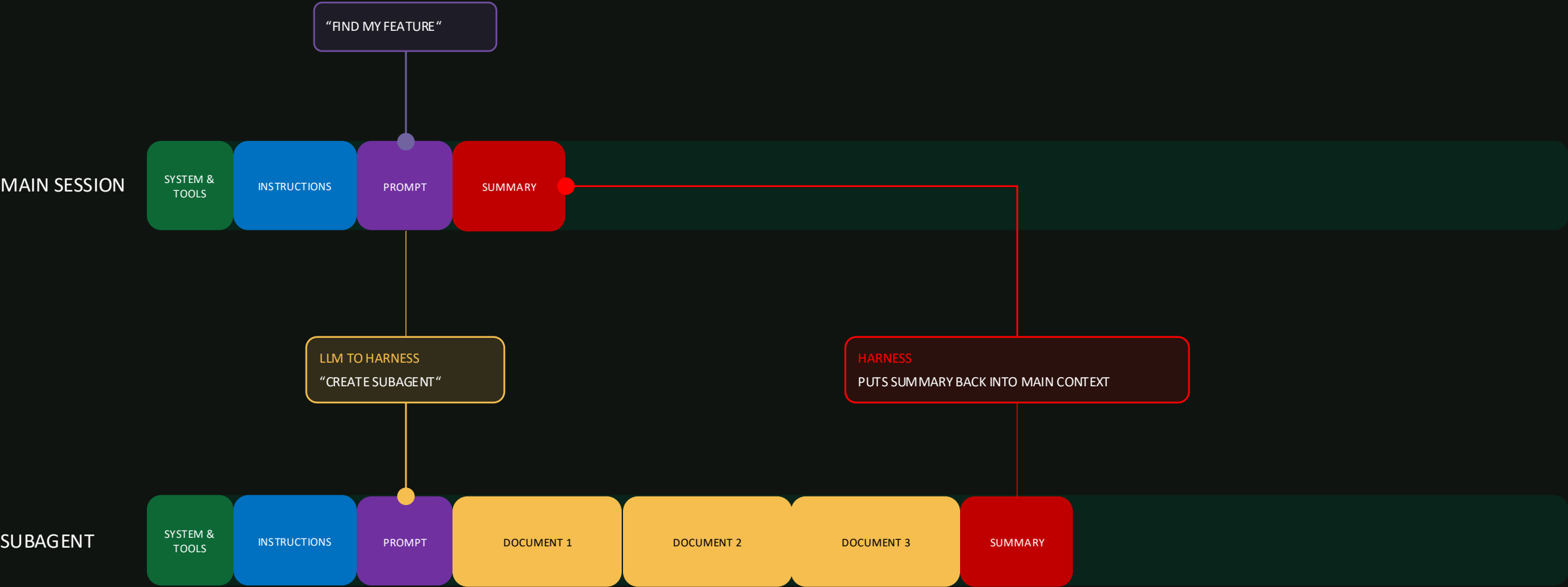
Skills



MCPs



Subagents



Other Agent Configs

 Scoped Instructions Conditional Instructions based on file path patterns. None-deterministic – offered to the Agent same as Skills.	 Prompt Files Manually invoked prompts. Can be used to trim Toolset and Invoke Custom Agents. Use them as “manual starting point” to kick off Custom agents and / or Skills with a common prompt.	 Copilot Memory Automated, small “always-on” instructions. Used across Copilot Surfaces uniformly. Makes sense to regularly check them.
--	--	---

Power User Guidance

These Tips require good **knowledge and time-invest**, as they are conditional and / or come with trade-offs.

Think in Code

Prefer creating scripts to analyze files over feeding it to AI.

Consider CLIs vs. MCPs

CLI Tools can be more optimal in certain scenarios due to less static tokens.

Improve Shell Outputs

Shell outputs can be extremely long – use something like <https://github.com/rtk-ai/rtk> to reduce their outputs.

Run “/chronicle tip” regularly

Analyze your usage in Copilot CLI to find improvement areas.

Collapse Tool Calls

Using something like <https://github.com/jsturtevant/copilot-codeact-plugin> can help collapse multiple tool calls into one.

Model Specific Context Optimization

Models behave different and can be tweaked.



Important Future Traits

 Build your Analytical Skills Coding was never the true value of developers – it was their analytical skills and the ability to become proficient in any domain quickly. Being able to tell an Agent precisely what to do, in the speak of the domain, will become the most valuable.	 Apply Good Architecture Domain Driven Design, Hexagonal Architecture, CQRS, Event Driven Design... Allows easier agent discovery. Gives stronger guardrails, avoids excess agent session (fixing, debugging, errors etc.).	 Iterate on Prompts & Agent Configs Approach AI Agents with an engineering mind. Keep configs fresh, treat agent misses like incidents. Use <code>`/chronicle`</code> in the CLI regularly to understand improvement areas.
---	--	---

summary

5 THINGS YOU CAN START DOING TODAY

- ✓ Chose the right model for the right task.
- ✓ Provide clear guidance in your prompts.
- ✓ Research – Plan – Implement.
- ✓ Provide deterministic guardrails (tests, linters, security scans etc.).
- ✓ Maintain a concise, human-written copilot-instructions.md. Use it as agent-miss log & to trim outputs.





Thank you